

Unsupervised learning

Now the labels y are missing. Only given x , we want to find structures/patterns in the data

	Is the missing variable continuous? Then we want to find subspaces	Is the missing variable discrete? Then we want to find clusters (i.e. Clustering)
Probabilistic	ICA	GMM(Gaussian Mixture Model) ~ EM
Deterministic	PCA	K-means

K-means

Data: $\{x^{(1)}, x^{(2)}, \dots, x^{(n)}\}$, $x^{(i)} \in \mathbb{R}^d$

From this data, we learn parameters which are k different centroids: $\{\mu_1, \dots, \mu_k\}$, $\mu_j \in \mathbb{R}^d$
(k is a hyperparameter)

The loss function is defined by the “distortion function”: $J(c, \mu) = \sum_{i=1}^n \|x^{(i)} - \mu_{c^{(i)}}\|_2^2$

We can say optimizing = minimizing the value of the loss function. We start by initializing the centroid, by randomly picking k data points and setting them as the initial centroids. Then we use coordinate descent: we fix c and optimize μ , and vice versa. By this method the distortion function's value will only go down or stay the same (monotonically decreasing) and has a lower bound of zero, so the algorithm is guaranteed to converge.

```
Repeat until convergence: {
  for every i, set
     $c^{(i)} = \operatorname{argmin}_j \|x^{(i)} - \mu_j\|_2^2$ 
    // assign  $x^{(i)}$  their centroid membership

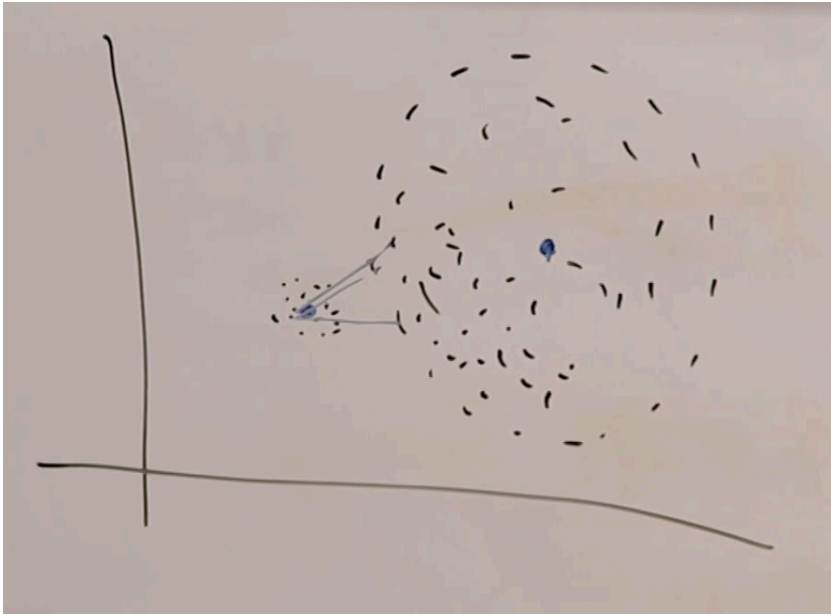
  for every j, set
    
$$\mu_j = \frac{\sum_{i=1}^n 1\{c^{(i)}=j\} x^{(i)}}{\sum_{i=1}^n 1\{c^{(i)}=j\}}$$

    // recalculate the centroids to the mean of all its  $x^{(i)}$  members)
```

Remark

- The algorithm could be alternating between different centroids that produce the same loss, but in real world cases this is almost never likely to happen.
- While k-means is guaranteed to converge, it is not guaranteed to converge to the same answer (centroids) always. Different initializations can lead to different answers, hence it isn't a globally convex objective.
- How do we choose the right k?
 - Data driven way: we allocate some data to the validation set, and train the model with the training set and compute the loss on both the training set and the validation set (don't train on the validation set). As we increase k and both of the losses will only go down. However there will be a point where the loss on the validation set plateaus or even goes up, and that's the point we decide it's overfitting, so we may want to stop there.
 - Intuition: Depending on the problem setting and data at hand we can apply domain knowledge/heuristics to decide how k should be oriented. Generally we don't want k to be too big since we are looking for some kind of simplification ('most' cases).

Limitations



Problems can arise when the data clusters are not similar-sized spheres

Expectation Maximization Algorithm

Classifying ML Algorithms with how they view the data (Supervised, Unsupervised)

Probabilistic: assumes data was generated by some distribution ($\sim$); do MLE

- Discriminative: likelihood = $p_{\theta}(y | x)$
 - GLMs
- Generative: likelihood = $p_{\theta}(y, x) = p(y) \cdot p_{\theta}(x | y)$ [$p_{\theta}(x)$ for unsupervised]
 - GDAs, Naive Bayes
 - EMs (GMM), ICA*, VAE, Diffusion

Deterministic: no distributional assumption, optimizes a criterion directly

- Decision Trees, k-NN, SVM, Perceptron, k-means, PCA

*Neural nets ($\ni$ CNN, RNN, Transformer, etc) is an architecture, so they appear under whichever objective is placed on top, hence not a category here

For unsupervised learning (no y), we introduce a latent variable z in place of y as a modeling

device, and we model $p_{\theta}(x) = \sum_z p_{\theta}(x, z) = \sum_z p_{\theta}(x | z) p_{\theta}(z) \quad // \quad p_{\theta}(x) = p(x; \theta)$

Again, assuming all the examples are IID,

$$\begin{aligned} \operatorname{argmax}_{\theta} \prod_{i=1}^n p_{\theta}(x^{(i)}) &= \operatorname{argmax}_{\theta} \log\left(\prod_{i=1}^n p_{\theta}(x^{(i)})\right) = \operatorname{argmax}_{\theta} \sum_{i=1}^n \log(p_{\theta}(x^{(i)})) \\ &= \operatorname{argmax}_{\theta} \sum_{i=1}^n \log\left(\sum_z p_{\theta}(x^{(i)}, z)\right) = \operatorname{argmax}_{\theta} \sum_{i=1}^n \log\left(\sum_z p_{\theta}(x^{(i)} | z) p_{\theta}(z)\right) \end{aligned}$$

However $\log\left(\sum_z p_{\theta}(x^{(i)} | z) p_{\theta}(z)\right)$ is not easy to be an objective for optimization since

- 1) the log sits outside the sum, so it doesn't separate and there's no closed-form optimum
- (2) $\sum_z$ can be intractable to even compute when z is high-dimensional
- (3) it's non-convex, so gradient methods aren't guaranteed to find the global maximum.

Hence the EM(Expectation Maximization): The optimization hack - it fixes 1

Jensen's Inequality

For a random variable x , when f is a convex function: $E[f(x)] \geq f[E(x)]$
 (Concave: $E[f(x)] \leq f[E(x)]$)

remark

- equal when x is a constant(or can be treated like a constant) or f is linear on the relevant range

Looking at only a single example,

$$\begin{aligned} \log(p_\theta(x)) &= \log\left(\sum_z p_\theta(x, z)\right) = \log\left(\sum_z Q(z) \cdot \frac{p_\theta(x, z)}{Q(z)}\right) \quad // \sum_z Q(z) = 1, Q(z) > 0 \\ &= \log E_{z \sim Q}\left[\frac{p_\theta(x, z)}{Q(z)}\right] \end{aligned}$$

Since $\log$ is concave, applying the Jensen's inequality gives us

$$\log E\left[\frac{p_\theta(x, z)}{Q(z)}\right] \geq E\left[\log\left(\frac{p_\theta(x, z)}{Q(z)}\right)\right] := ELBO(\text{evidence lower bound})$$

So $\log(p_\theta(x)) \geq ELBO$.

Now if $\frac{p_\theta(x, z)}{Q(z)}$ is a constant, then $\log(p_\theta(x)) = ELBO$.

$$\frac{p_\theta(x, z)}{Q(z)} = c \rightarrow \frac{p_\theta(x, z)}{c} = Q(z) \rightarrow \sum_z \frac{p_\theta(x, z)}{c} = 1 \rightarrow c = \sum_z p_\theta(x, z)$$

$$\text{So for } \frac{p_\theta(x, z)}{Q(z)} \text{ to be a constant, } Q(z) = \frac{p_\theta(x, z)}{\sum_z p_\theta(x, z)} = \frac{p_\theta(x, z)}{p_\theta(x)} = p_\theta(z | x)$$

Now the general EM algorithm:

```
Repeat until convergence: {
    //E-Step(fix theta)
    for each i(i in examples):
         $Q_i(z^{(i)}) := p_\theta(z^{(i)} | x^{(i)})$ 

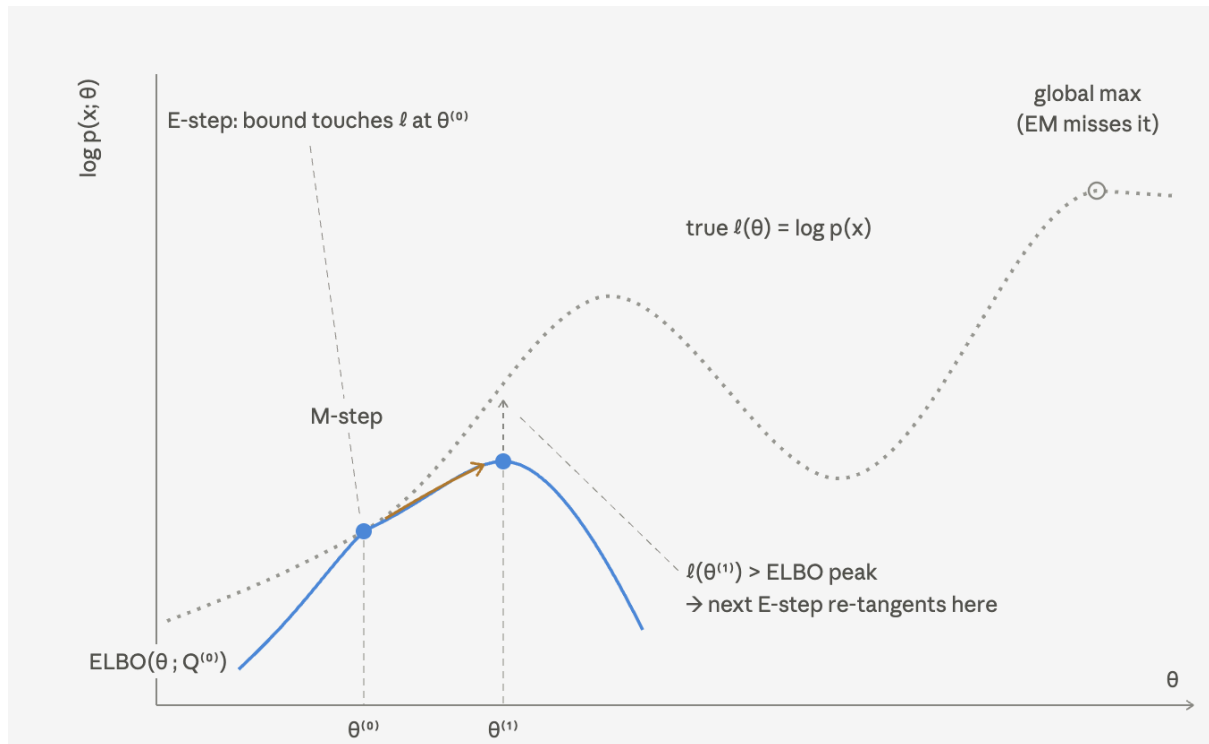
    // M-Step(fix Q(z))
     $\theta := \operatorname{argmax}_\theta \sum_{i=1}^n ELBO(x^{(i)}; Q_i(z^{(i)}), \theta) = \operatorname{argmax}_\theta \sum_{i=1}^n E\left[\log\left(\frac{p_\theta(x^{(i)}, z^{(i)})}{Q_i(z^{(i)})}\right)\right]$ 
     $= \operatorname{argmax}_\theta \sum_{i=1}^n \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{p_\theta(x^{(i)}, z^{(i)})}{Q_i(z^{(i)})}$ 
}
```

E-Step: For the given θ (fix θ), what is the choice of Q that will make ELBO equal to the log probability?)

M-Step: With that fixed $Q_i(z^{(i)})$ value, what's the θ that maximizes $\sum_{i=1}^n \log(p_\theta(x^{(i)}))$?

And convergence can be defined as when the ELBO almost plateaus

Guarantee of convergence:



$$l(\theta^{(t+1)}) \geq ELBO(Q^{(t)}, \theta^{(t+1)}) \geq ELBO(Q^{(t)}, \theta^{(t)}) = l(\theta^{(t)})$$

$$\text{so } l(\theta^{(t+1)}) \geq l(\theta^{(t)})$$

EM on Gaussian Mixture Models

$$z^{(i)} \text{ (latent variable)} \sim \text{Multinomial}(\phi) \quad \phi_j \geq 0, \sum_{j=1}^k \phi_j = 1$$

$$x^{(i)} | z^{(i)} = j \sim N(\mu_j, \Sigma_j) \quad \mu_j \in R^d, \Sigma_j \in R^{d \times d}$$

$$(\theta = \phi, \mu, \Sigma)$$

Repeat until convergence: {

//E-step

for each i:

$$w_j^{(i)} = \frac{\frac{1}{(2\pi)^{d/2} \det(\Sigma_j)^{1/2}} e^{(-\frac{1}{2}(x^{(i)} - \mu_j)^T \Sigma_j^{-1} (x^{(i)} - \mu_j))} \cdot \phi_j}{\sum_{j=1}^k \left(\frac{1}{(2\pi)^{d/2} \det(\Sigma_j)^{1/2}} e^{(-\frac{1}{2}(x^{(i)} - \mu_j)^T \Sigma_j^{-1} (x^{(i)} - \mu_j))} \cdot \phi_j \right)}$$

//M-step

$$\mu_j = \frac{\sum_{i=1}^n w_j^{(i)} \cdot x^{(i)}}{\sum_{i=1}^n w_j^{(i)}}$$

$$\phi_j = \frac{1}{n} \sum_{i=1}^n w_j^{(i)}$$

$$\Sigma_j = \frac{\sum_{i=1}^n w_j^{(i)} \cdot (x^{(i)} - \mu_j) \cdot (x^{(i)} - \mu_j)^T}{\sum_{i=1}^n w_j^{(i)}}$$

}

Remark

- $w_j^{(i)}$ is called responsibility - how much the cluster j is responsible for example i
- Everything in the M-step is just a weighted version of the fully-observed MLE(GDA). If the labels were observed, we'd compute for cluster j: the mean of the points in j, the covariance of the points in j, and the fraction of points in j. Since labels aren't observed, we replace the hard indicator $1\{z^{(i)} = j\}$ with the soft weight $w_j^{(i)}$:

- μ_j = weighted mean of the $x^{(i)}$, weighted by $w_j^{(i)}$
- Σ_j = weighted covariance around that new μ_j
- ϕ_j = the share of total responsibility that cluster j holds

So the M-step formulas aren't new machinery but the ordinary Gaussian MLE with $1\{z^{(i)} = j\} \rightarrow w_j^{(i)}$

(Full Derivation)

Repeat until convergence: {

//E-step

for each i:

$$\begin{aligned} w_l^{(i)} &= Q_i(z^{(i)} = l) = p_\theta(z^{(i)} = l | x^{(i)}) = \frac{p_\theta(x^{(i)} | z^{(i)} = l) \cdot p_\theta(z^{(i)} = l)}{\sum_j p_\theta(x^{(i)} | z^{(i)} = j) \cdot p_\theta(z^{(i)} = j)} \\ &= \frac{\frac{1}{(2\pi)^{d/2} \det(\Sigma_l)^{1/2}} e^{(-\frac{1}{2}(x^{(i)} - \mu_l)^T \Sigma_l^{-1} (x^{(i)} - \mu_l))} \cdot \phi_l}{\sum_{j=1}^k \left(\frac{1}{(2\pi)^{d/2} \det(\Sigma_j)^{1/2}} e^{(-\frac{1}{2}(x^{(i)} - \mu_j)^T \Sigma_j^{-1} (x^{(i)} - \mu_j))} \cdot \phi_j \right)} \end{aligned}$$

//M-step

Given $Q_i(z^{(i)} = l) = w_l^{(i)} \in R$, update the parameters to

$$\begin{aligned} & \operatorname{argmax}_\theta \sum_{i=1}^n \sum_{j=1}^k Q_i(z^{(i)} = j) \log \frac{p_\theta(z^{(i)} = j, x^{(i)})}{Q_i(z^{(i)} = j)} \\ &= \operatorname{argmax}_\theta \sum_{i=1}^n \sum_{j=1}^k Q_i(z^{(i)} = j) \log \frac{p_\theta(x^{(i)} | z^{(i)} = j) \cdot p_\theta(z^{(i)} = j)}{Q_i(z^{(i)} = j)} \\ &= \operatorname{argmax}_\theta \sum_{i=1}^n \sum_{j=1}^k w_j^{(i)} \log \frac{p_\theta(x^{(i)} | z^{(i)} = j) \cdot p_\theta(z^{(i)} = j)}{w_j^{(i)}} \\ &= \operatorname{argmax}_\theta \sum_{i=1}^n \sum_{j=1}^k w_j^{(i)} \log \frac{\frac{1}{(2\pi)^{d/2} \det(\Sigma_j)^{1/2}} e^{(-\frac{1}{2}(x^{(i)} - \mu_j)^T \Sigma_j^{-1} (x^{(i)} - \mu_j))} \cdot \phi_j}{w_j^{(i)}} \end{aligned}$$

Updating for μ_l

$$\begin{aligned} \nabla_{\mu_l} \sum_{i=1}^n \sum_{j=1}^k w_j^{(i)} \log \frac{\frac{1}{(2\pi)^{d/2} \det(\Sigma_j)^{1/2}} e^{(-\frac{1}{2}(x^{(i)} - \mu_j)^T \Sigma_j^{-1} (x^{(i)} - \mu_j))} \cdot \phi_j}{w_j^{(i)}} \\ &= \nabla_{\mu_l} \sum_{i=1}^n \sum_{j=1}^k w_j^{(i)} \cdot \left(-\frac{1}{2} (x^{(i)} - \mu_j)^T \Sigma_j^{-1} (x^{(i)} - \mu_j) \right) \\ &= -\frac{1}{2} \sum_{i=1}^n w_l^{(i)} \cdot 2 \Sigma_j^{-1} \cdot (x^{(i)} - \mu_j) \cdot -1 \\ &= \sum_{i=1}^n w_l^{(i)} \cdot \Sigma_l^{-1} (x^{(i)} - \mu_l) \end{aligned}$$

set this to zero and solve for μ_l , then we get:

$$\mu_l := \frac{\sum_{i=1}^n w_l^{(i)} \cdot x^{(i)}}{\sum_{i=1}^n w_l^{(i)}} \quad // \Sigma_j^{-1} \text{ cancelled out}$$

Updating for ϕ_l

$$\nabla_{\phi_l} \sum_{i=1}^n \sum_{j=1}^k w_j^{(i)} \log \frac{\frac{1}{(2\pi)^{d/2} \det(\Sigma_j)^{1/2}} e^{(-\frac{1}{2}(x^{(i)} - \mu_j)^T \Sigma_j^{-1} (x^{(i)} - \mu_j))} \cdot \phi_j}{w_j^{(i)}}$$

$$= \nabla_{\phi_l} \sum_{i=1}^n \sum_{j=1}^k w_j^{(i)} \cdot \log \phi_j$$

$$\text{Construct lagrangian } \mathcal{L}(\phi) = \sum_{i=1}^n \sum_{j=1}^k w_j^{(i)} \cdot \log \phi_j + \beta (\sum_{j=1}^k \phi_j - 1)$$

$$\nabla_{\phi_l} \mathcal{L}(\phi) = \sum_{i=1}^n \frac{w_l^{(i)}}{\phi_l} + \beta$$

set this to zero and solve for ϕ_l , then we get :

$$\phi_l := \frac{\sum_{i=1}^n w_l^{(i)}}{-\beta}$$

using the constraint that $\sum_{j=1}^k \phi_j = 1$, we easily find that $-\beta = \sum_{i=1}^n \sum_{j=1}^k w_j^{(i)} = \sum_{i=1}^n 1 = n$.

$$\phi_l := \frac{1}{n} \sum_{i=1}^n w_l^{(i)}$$

Updating for Σ_l

$$\nabla_{\Sigma_l} \sum_{i=1}^n \sum_{j=1}^k w_j^{(i)} \log \frac{\frac{1}{(2\pi)^{d/2} \det(\Sigma_j)^{1/2}} e^{(-\frac{1}{2}(x^{(i)} - \mu_j)^T \Sigma_j^{-1} (x^{(i)} - \mu_j))} \cdot \phi_j}{w_j^{(i)}}$$

$$= \nabla_{\Sigma_l} \sum_{i=1}^n w_l^{(i)} \cdot [-\frac{1}{2} \log \det(\Sigma_l) - \frac{1}{2} (x^{(i)} - \mu_l)^T \Sigma_l^{-1} (x^{(i)} - \mu_l)]$$

$$= \sum_{i=1}^n w_l^{(i)} [-\frac{1}{2} \Sigma_l^{-1} + \frac{1}{2} \Sigma_l^{-1} (x^{(i)} - \mu_l)(x^{(i)} - \mu_l)^T \Sigma_l^{-1}]$$

set this to zero and solve for Σ_l , then we get:

$$\Sigma_l := \frac{\sum_{i=1}^n w_l^{(i)} \cdot (x^{(i)} - \mu_l)(x^{(i)} - \mu_l)^T}{\sum_{i=1}^n w_l^{(i)}}$$

}

$$z^{(i)} = j \mid x^{(i)}$$

$$w_1^{(i)}$$

$$w_2^{(i)}$$

$$\dots$$

$$w_k^{(i)}$$

$$z^{(i)} \sim \text{Multinomial}(\phi)$$

$$\phi = \{\phi_1, \phi_2, \dots, \phi_k\}$$

$$\phi_2$$

$$\dots$$

$$\phi_k\}$$

$$x^{(i)} \mid z^{(i)} = j \sim N(\mu_j, \Sigma_j)$$

$$\mu = \{\mu_1, \mu_2, \dots, \mu_k\}$$

$$\mu_2$$

$$\dots$$

$$\mu_k\}$$

$$\begin{pmatrix} 1 \times d \end{pmatrix}$$

$$\begin{pmatrix} 1 \times d \end{pmatrix}$$

$$\begin{pmatrix} 1 \times d \end{pmatrix}$$

$$\Sigma = \{\Sigma_1, \Sigma_2, \dots, \Sigma_k\}$$

$$\Sigma_2$$

$$\Sigma_k\}$$

$$\begin{pmatrix} 1 \times d \times d \end{pmatrix}$$

$$\begin{pmatrix} 1 \times d \times d \end{pmatrix}$$

$$\begin{pmatrix} 1 \times d \times d \end{pmatrix}$$

Multivariate
Gaussian

Multivariate
Gaussian

Multivariate
Gaussian

PCA (Principal Component Analysis)

$X \in \mathbb{R}^{n \times d}$ Assume X is normalized $\left(\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij}, \sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \mu_j)^2} \right)$
 $a_{ij} \leftarrow \frac{x_{ij} - \mu_j}{\sigma_j}$

cov matrix $\frac{1}{n} X^T X = V L V^T$
 Diagonalize

• Since $X^T X$ is symmetric, $V V^T = I$

• $L = \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_d \end{pmatrix}$

Re-arrange V, L s $\lambda_1 \geq \lambda_2 \geq \dots \lambda_d \geq 0$

pick a k ($< d$) that will be the reduced dimension

put together k eigenvectors that correspond to the k largest eigenvalues (vertically) into a matrix

$\tilde{V} = \begin{pmatrix} | & | & \dots & | \\ | & | & \dots & | \\ | & | & \dots & | \\ | & | & \dots & | \end{pmatrix}^d$

(This can be thought of as the 'parameter' we're learning.)

This is the basis vector for our new subspace

Then we get $\tilde{X} = X \tilde{V} \in \mathbb{R}^{n \times k}$

→ We're trying to move all data from $\mathbb{R}^d$ into a k -dimensional subspace. How?

* Interpretation 1: Data's variance should be maximized after moving
s.t. X is only moved with XA (i.e. $A^T X^T$) and $AA^T = I_k$

How do we quantify the variance of $\begin{pmatrix} X \\ A \end{pmatrix}$? → Sum of k features' individual variances
= trace (Cov matrix $\frac{1}{n} A^T X X^T A \in \mathbb{R}^{k \times k}$)

$$\begin{pmatrix} d \\ X \end{pmatrix} \begin{pmatrix} k \\ A \end{pmatrix} = \begin{pmatrix} k \\ XA \end{pmatrix} \quad \begin{matrix} X^{(1)} \\ \vdots \\ X^{(n)} \end{matrix} \quad \begin{matrix} A_1 \sim A_k \end{matrix} \quad \begin{matrix} X^{(1)}A_1 & \dots & X^{(1)}A_k \\ \vdots & & \vdots \\ X^{(n)}A_1 & \dots & X^{(n)}A_k \end{matrix}$$

$$\frac{1}{n} \begin{pmatrix} n \\ X^{(1)}A_1 \dots X^{(n)}A_1 \end{pmatrix} \begin{pmatrix} k \\ X^{(1)}A_1 & \dots & X^{(1)}A_k \\ \vdots & & \vdots \\ X^{(n)}A_1 & \dots & X^{(n)}A_k \end{pmatrix} = \begin{pmatrix} \frac{1}{n} \sum_{i=1}^n (X^{(i)}A_1)^2 & \dots & \frac{1}{n} \sum_{i=1}^n (X^{(i)}A_1 X^{(i)}A_k) \\ \vdots & & \vdots \\ \frac{1}{n} \sum_{i=1}^n (X^{(i)}A_k X^{(i)}A_1) & \dots & \frac{1}{n} \sum_{i=1}^n (X^{(i)}A_k)^2 \end{pmatrix}$$

so $\arg\max_A \text{trace}(\frac{1}{n} A^T X X^T A)$, s.t. $AA^T = I_k$. Denote $\frac{1}{n} X^T X$ as Σ

→ Construct Lagrangian $\mathcal{L}(A, B) = \text{tr}(A^T \Sigma A) - B \cdot \text{tr}(A^T A - I_k)$

$$\frac{\partial}{\partial A} \mathcal{L}(A, B) = 2 \Sigma A - 2 A B = 0 \Rightarrow \Sigma A = A B \Rightarrow A^T \Sigma A = B$$

so $\arg\max_A (\dots)$ is when $A^T \Sigma A = B$, and problem becomes $\arg\max_A \text{tr}(B)$

What is B ?

$$\begin{pmatrix} d \\ \Sigma = \frac{1}{n} X^T X \end{pmatrix} \begin{pmatrix} k \\ A \end{pmatrix} = \begin{pmatrix} k \\ A \end{pmatrix} \begin{pmatrix} k \\ B \end{pmatrix}$$

Since at our solution $B = A^T \Sigma A$ is a symmetric matrix, it is diagonalizable.

Let $B = \begin{pmatrix} \lambda_{B1} & & 0 \\ & \ddots & \\ 0 & & \lambda_{Bk} \end{pmatrix}$, then $\Sigma A_j = \lambda_{Bj} A_j$, so $\lambda_{B1} \dots \lambda_{Bk}$ turn out to be Σ 's eigenvalues and A 's columns turn out to be Σ 's eigenvectors.

So $B = \text{diag}(k \text{ eigenvalues picked from } \Sigma\text{'s } d \text{ eigenvalues})$ & $A = k \text{ corresponding eigenvectors of } \Sigma$

$\text{tr}(B) = \text{sum of whichever } k \text{ eigenvalues we picked, so pick the highest } k \text{ values}$

$\arg\max_A \text{tr}(B) = \Sigma\text{'s eigenvectors with the largest eigenvalues}$

⊕ How do we pick the right k ?

If $k=d$, we will capture all the variance = $\lambda_1 + \dots + \lambda_d$

$$\text{Variance captured} = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^d \lambda_j}$$

We want to use the least k to capture as much variance as possible

So set a percentage criterion, and increase k until you get it

* Interpretation 2: The data points' movement should be at the minimum

$$\begin{matrix} X & \xrightarrow{\cdot A} & XA & \xrightarrow{\cdot A^T} & XAA^T = \hat{X} \\ \mathbb{R}^{n \times d} & & \mathbb{R}^{n \times k} & & \mathbb{R}^{n \times d} \end{matrix}$$

Since $AA^T \neq I_d$, $X \neq \hat{X}$. so what's $\arg\min_A \|X - \hat{X}\|_F^2$ s.t. $A^T A = I_k$?

First, note that $X - \hat{X} = X - XAA^T = R$, then $RA = XA - XAA^T A = XA - XA = 0$.

So R 's each row (each example's difference) is orthogonal to A 's every column vector.

for each example, $x = \hat{x} + r$, and $\hat{x} \perp r$

$$\Rightarrow \|x\|_2^2 = \|\hat{x}\|_2^2 + \|r\|_2^2, \|\hat{x}\|_2^2 = \|xAA^T\|_2^2 = xAA^TAA^T x^T = \|xA\|_2^2$$

$$\text{So } \|x\|_2^2 = \|xA\|_2^2 + \|r\|_2^2$$

$$\text{So } \|X - \hat{X}\|_F^2 = \sum_{i=1}^n \|x^{(i)}\|_2^2 = \sum_{i=1}^n \|x^{(i)}A\|_2^2 + \sum_{i=1}^n \|r^{(i)}\|_2^2$$

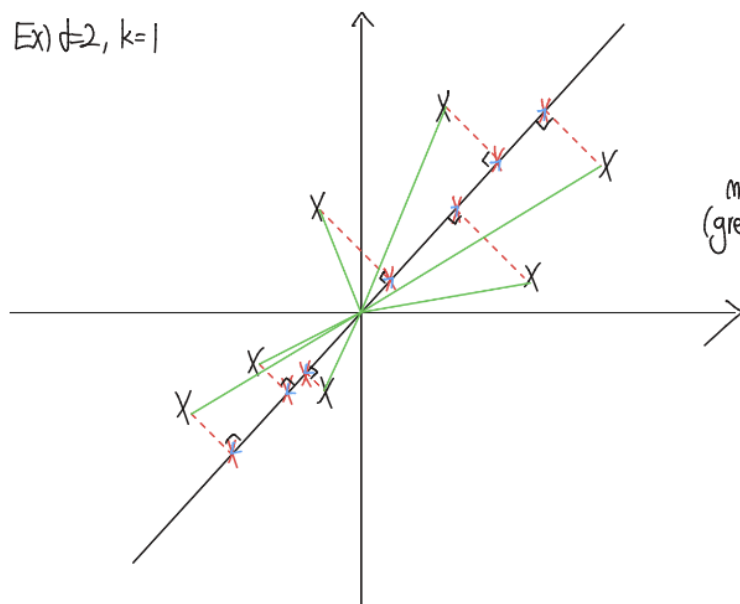
$$\Rightarrow \frac{1}{n} \|X - \hat{X}\|_F^2 = \frac{1}{n} \sum_{i=1}^n \|x^{(i)}\|_2^2 = \frac{1}{n} \sum_{i=1}^n \|x^{(i)}A\|_2^2 + \frac{1}{n} \sum_{i=1}^n \|r^{(i)}\|_2^2$$

$$\begin{matrix} \text{~~~~~} & \text{~~~~~} & \text{~~~~~} \\ \text{tr}(\Sigma) & \text{tr}(A^T \Sigma A) & \text{Reconstruction Error (MSE)} \\ = \text{constant} & & \text{(The variance we threw away)} \\ \text{(determined by } X) & & \end{matrix} \quad \text{Reconstruction Error (MSE)} = \frac{1}{n} \|X - \hat{X}\|_F^2$$

$$\text{So } \arg\min_A \|X - \hat{X}\|_F^2 = \arg\max_A (\text{tr}(A^T \Sigma A))$$

$\therefore$ Same as Interpret. 1 (Maximizing Variance)

Ex) $d=2, k=1$



$$\begin{matrix} \frac{1}{n} \sum_{i=1}^n \|x^{(i)}\|_2^2 = \frac{1}{n} \sum_{i=1}^n \|x^{(i)}A\|_2^2 + \frac{1}{n} \sum_{i=1}^n \|r^{(i)}\|_2^2 \\ \text{~~~~~} & \text{~~~~~} & \text{~~~~~} \\ \text{tr}(\Sigma) & \text{tr}(A^T \Sigma A) & \text{Reconstruction Error (MSE)} \\ \text{mean of all the} & \text{mean of all the} & \text{mean of all the} \\ \text{(green line's length)}^2 & \text{(blue line's length)}^2 & \text{(red line's length)}^2 \end{matrix}$$

PCA vs Lasso

	Lasso	PCA
Goal	Which features predict y ?	What redundancy/correlation exists among features?
y	needed (supervised)	not needed (unsupervised)
output	each column makes it or doesn't (sparse)	all columns make it, but by different amounts

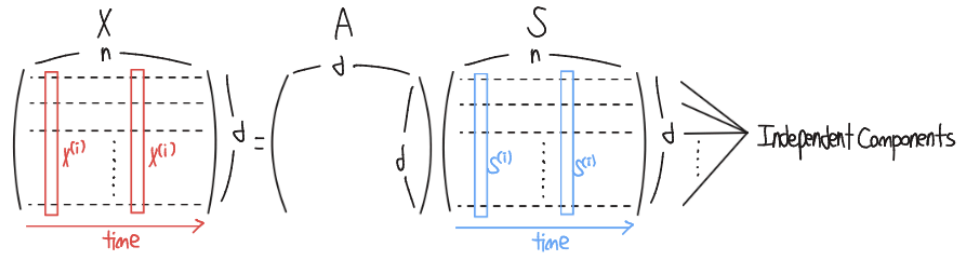
- Lasso SELECTS original features (they keep their meaning) while PCA MIXES them (each v_i is a weighted combination of all d features, so new axes have no original feature meaning)
- Both reduce dimensionality, but they're complementary: PCA before you have a target, Lasso once you do

ICA (Independent Component Analysis)

$$x^{(i)} = A s^{(i)} \quad * i = \text{time}$$

data we see mixing matrix source

Problem: What is $A^{-1} = W$?



(ICA Ambiguities)

- Scale: $x = As = (\frac{A}{c})(cs)$
 $\Rightarrow$ solution: normalize s
- Permutation: $x = As = (AP^{-1})(Ps)$ // $S_1 \leftrightarrow S_2, A_1 \leftrightarrow A_2$: same result
 $\Rightarrow$ no problem since it's just the order of the independent components changing
- Orthogonality & Gaussian data: $x \sim N(0, AA^T)$
 $x' = ARs$ (R = orthogonal matrix, $R^T R = RR^T = I$)
 $= A's$
 $x' \sim N(0, A'A'^T)$
 $\sim N(0, AA^T)$

$\Rightarrow$ Can't tell difference between many (A, S) possibilities $\Rightarrow$ ICA requires S to be non-Gaussian
 (Not source-separated) (At most one component may be gaussian)

Assume S_j follows non-Gaussian distribution $p_S(s_j)$

$\hookrightarrow$ Set CDF as a sigmoid: $g_S(s_j) = \frac{1}{1+e^{-s_j}}$ $p_S(s_j) = g'_S(s_j)$ (Any non-gaussian CDF will work)

$$p(S) = \prod_{j=1}^d p_S(s_j) \quad // \text{ Since we're assuming the sources are independent}$$

$$\text{since } s = Wx, \quad p(x) = \prod_{j=1}^d p_S(w_j^T x) \cdot |\det(W)|$$

$$\text{So } \mathcal{L}(W) = \sum_{i=1}^n \left(\sum_{j=1}^d [\log g'_S(w_j^T x^{(i)})] + \log |\det(W)| \right)$$

$\Rightarrow$ Randomly init W , do SGD

$$W := W + \alpha \frac{\partial \mathcal{L}(W)}{\partial W}$$

$$:= W + \alpha \begin{pmatrix} 1 - 2g(w_1^T x^{(i)}) \\ \vdots \\ 1 - 2g(w_d^T x^{(i)}) \end{pmatrix} x^{(i)T} + (W^T)^{-1}$$